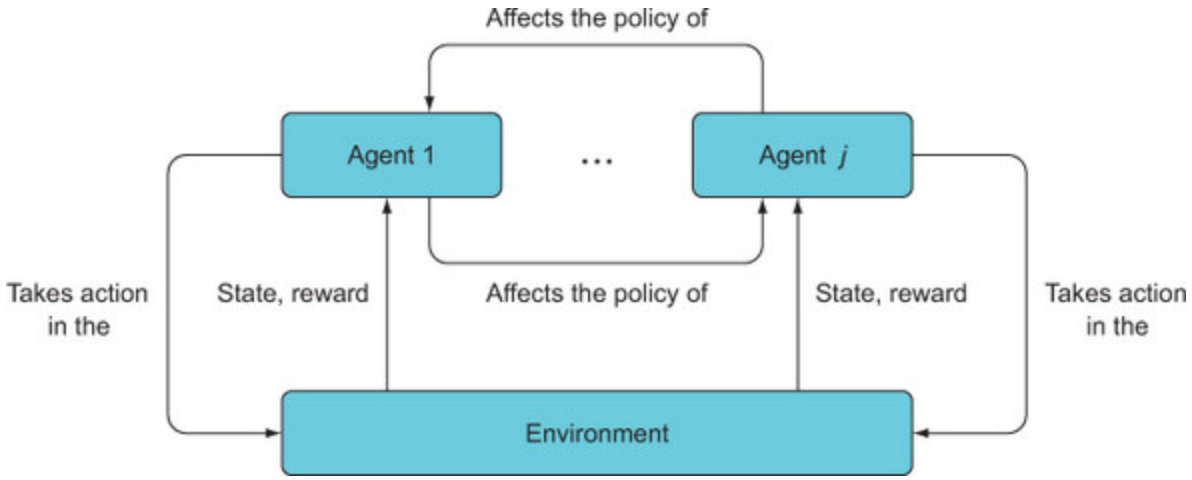
patterns in the state of the environment that change over time. Our DQN is not directly aware of the actions of the other players. This is not an issue because these NPCs do not learn; they have fixed policies. As you'll see in this chapter, sometimes we want to go beyond mere NPCs and actually model the behavior of many interacting agents that learn (figure 9.1), and this requires a bit of a reformulation of the basic reinforcement learning framework you've learned about so far in this book.

Figure 9.1. In the multi-agent setting, each agent's actions not only affect the evolution of the environment, but also the policies of other agents, leading to highly dynamic agent interactions. The environment will produce a state and reward, which each agent 1 through j use to take actions using their own policies. However, each agent's policy will affect all the other agents' policies.



For example, imagine that we directly want to control the actions of many interacting agents in some environment using a deep reinforcement learning algorithm. For example, there are games with multiple players grouped into teams, and we may want to develop an algorithm that can play a bunch of players on a team against another team. Or we may want to control the actions of hundreds of simulated cars to model traffic patterns. Or maybe we're economists and we want to model the behavior of thousands of agents in a model of an economy. This is a different situation than having NPCs because, unlike NPCs, these other agents all learn, and their learning is affected by each other.

The most straightforward way to extend what we know already into a multi-agent setting is to instantiate multiple DQNs (or some other similar